

Information Retrieval Systems on GamingStackExchange.com



M.Tech (Artificial Intelligence), CSE
Information Retrieval

Name: Rahul Pargi

Enrollment No.: 25PCS016

Professor: Dr. Dwijen Rudrapal

Department of Computer Science and Engineering
National Institute of Technology, Agartala

India - 799046

September 25, 2025

Abstract

This project implements and evaluates two fundamental Information Retrieval (IR) systems on the **Gaming StackExchange.com** dataset: the Boolean Retrieval Model and the Term-Frequency Inverted Index Model. The goal is to analyze the effectiveness of these systems in retrieving relevant answers for user queries.

The Boolean model offers simplicity and efficiency for exact keyword matching but struggles with nuanced or ambiguous queries due to its lack of ranking. In contrast, the TF-based Inverted Index leverages term frequency to rank results, thereby providing significantly improved precision and relevance for diverse queries. Experimental evaluation using metrics such as Precision@10 and nDCG@10 demonstrates that the TF-based system consistently outperforms Boolean retrieval for long and ambiguous queries, while Boolean retrieval remains useful for fast, exact-match searches.

Overall, the results highlight the importance of ranking mechanisms in modern IR systems and provide insights into how simple models scale to community-driven Q&A platforms.

Contents

1	Introduction	3
2	Objectives	3
3	Dataset and Queries	4
3.1	Dataset	4
3.2	Queries	4
4	Development Environment and Project Setup	5
4.1	Development Environment	5
4.2	Project Setup and Execution	6
5	Dataset Analysis	6
5.1	Analysis of Question 1: Common Terms in the Collection	6
5.2	Analysis of Question 2: Top Question Tags	8
5.3	Analysis of Question 3: Average Number of Words and Sentences	9
5.4	Analysis of Question 4: Answers per Question	9
5.5	Analysis of Question 5: Unanswered Questions	10
5.6	Analysis of Question 6: Accepted Answers and User Reputation	11
5.7	Analysis of Question 7: Readability and Its Impact on Answer Likelihood	12
5.8	Analysis of Question 8: Duplicate Questions Analysis	13
5.9	Analysis of Question 9: Analysis of Comments on Selected Questions	14
5.10	Analysis of Question 10: Independent Analysis – Correlation of Question Features with Time to First Answer	16
5.11	Analysis of Boolean Search System	17
5.12	Analysis of Term Frequency Inverted Index	18
6	Design and Implementation of Retrieval Models	19
6.1	Boolean Retrieval Model	19
6.2	Term-Frequency Inverted Index Model	20
7	Evaluation and Comparative Analysis of Retrieval Models	22
7.1	Evaluation Summary	22
7.2	Comparative Analysis of Retrieval Models	23
7.3	Key Insights	23
8	Final Conclusion	24

9	References	24
	References	24

1 Introduction

Information Retrieval (IR) focuses on building systems that can efficiently locate relevant information from large collections of unstructured text. With the rapid growth of online communities and user-generated content, designing effective retrieval mechanisms has become increasingly important.

This project investigates two foundational retrieval approaches:

- **Boolean Retrieval** – enables logical queries using operators such as **AND** and **OR**, providing simple and fast matching.
- **Term-Frequency Inverted Index** – constructs an index of terms with frequency counts and applies ranking strategies to enhance retrieval effectiveness.

The dataset used for experimentation is sourced from **Gaming StackExchange**, comprising more than 100,000 community-driven questions and answers related to game-play, troubleshooting, and platform-specific topics. Preprocessing steps such as lowercasing, punctuation removal, and stopword filtering were applied to ensure effective indexing and retrieval.

The evaluation focuses on both retrieval effectiveness and computational efficiency. A diverse query set was employed to highlight the practical trade-offs between Boolean matching and frequency-based ranking, providing insights into their relative strengths and limitations.

2 Objectives

The objectives of this project are to:

- Implement two contrasting IR systems: a Boolean retrieval model and a term-frequency-based inverted index.
- Preprocess and structure Gaming StackExchange data for indexing and retrieval.
- Evaluate both systems using a set of 20 representative queries.
- Assess performance using standard IR metrics, including **Precision@k**, **nDCG@k**, and average retrieval time.
- Compare the effectiveness and limitations of Boolean vs. ranking-based retrieval.
- Analyze all 10 assignment-specific questions to ensure comprehensive coverage and assessment.

3 Dataset and Queries

3.1 Dataset

The dataset used for this project was obtained from the official StackExchange data dump and consists of the following components:

- **Posts** – Questions and answers stored in XML format (`Posts.xml`). Each post contains its content along with embedded metadata such as creation dates, scores, accepted answer IDs, tags, owner IDs, view counts, and last activity dates.
- **Comments** – Comments associated with posts, stored in XML format (`Comments.xml`). Each comment includes its content and metadata such as creation dates, scores, user IDs, and the post ID it belongs to.

3.2 Queries

A set of **20 queries** was manually selected for evaluation. These queries cover a variety of topics related to Gaming StackExchange and represent realistic information needs. Each query is categorized by topic and length to ensure a diverse and comprehensive evaluation.

Query #	Query Text	Category	Length
1	Can I download PlayStation 3 games for my PlayStation 4	Gameplay	Medium
2	Playstation payment problem	Troubleshooting	Short
3	Downloading games onto a PlayStation 4	Gameplay	Medium
4	Dark Souls 2: Xbox360 vs PlayStation 3 vs Xbox One vs PlayStation 4	Gameplay	Long
5	GTA V won't load online	Troubleshooting	Short
6	Trouble downloading GTA5 on XBOX360	Troubleshooting	Medium
7	Is digital download GTA5 Faster than Physical copy?	Gameplay	Medium
8	Do I need to be online to play story mode?	Conceptual	Short
9	Do I need to play previous Witchers before Witcher3	Conceptual	Medium
10	Witcher 3 crashes constantly	Troubleshooting	Short
11	Skyrim reinstalling it with all the mods	Gameplay	Medium
12	Can I start with Dark souls 2?	Conceptual	Short
13	Defeating Moonlight Butterfly in Dark Souls	Gameplay	Medium
14	Max Farming in Dark Souls III	Gameplay	Medium

Query #	Query Text	Category	Length
15	What are the different endings of Sekiro?	Conceptual	Medium
16	How precise is the Deflect Mechanic in Sekiro?	Gameplay	Medium
17	Can you upgrade the horse in Elden Ring?	Gameplay	Short
18	Does dexterity increase weapon art speed in Elden Ring?	Conceptual	Medium
19	What happens if you defeat the Grafted Scion in the beginning of Elden Ring?	Conceptual	Long
20	I don't know where to go next in Elden Ring! How do I find out?	Gameplay	Long

4 Development Environment and Project Setup

4.1 Development Environment

The implementation and evaluation of the Information Retrieval systems were carried out in a Python-based environment. All experiments were performed on **Google Colab**, which provides a cloud-based platform with preconfigured Python runtimes and seamless library support.

The project requires the following software components and libraries:

- **Programming Environment:** Python 3 runtime in Google Colab.
- **Core Libraries:**
 - `numpy`, `pandas` – for data manipulation and analysis.
 - `matplotlib`, `wordcloud` – for visualization and graphical representation.
 - `scikit-learn`, `scipy` – for machine learning utilities, similarity computation, and statistical operations.
- **Data Processing Libraries:**
 - `lxml`, `MarkupSafe`, `notebook`, `openpyxl`, `Pillow`.
- **Text Processing Libraries:**
 - `nltk` – for natural language processing and stopword handling.
 - `textstat` – for computing readability scores and textual metrics.

This configuration ensures efficient preprocessing, indexing, and evaluation of large-scale XML datasets such as the StackExchange data dump.

4.2 Project Setup and Execution

To run the project and reproduce the analyses, please refer to the accompanying README file. The README provides step-by-step guidance for:

- Setting up the required software environment and dependencies.
- Installing necessary libraries and packages.
- Executing scripts for data analysis and generating results.

The complete project is available on GitHub at the following link:

https://github.com/Rahul-Pargi/IR_Project01

Following the instructions in the README ensures smooth execution and reproduction of all experiments and visualizations included in this report.

5 Dataset Analysis

5.1 Analysis of Question 1: Common Terms in the Collection

This analysis explores the most common terms used in the dataset of questions and answers. Word clouds are generated for the top-20 frequent words before and after removing stopwords. Additionally, the frequency distribution of words is plotted to examine if it follows Zipf’s law.



Figure 1: Top-20 Frequent Words (including stopwords)

After removing stopwords using NLTK, the Word Cloud changes significantly. Common function words such as “the”, “to” and “and”, which dominated the raw token cloud, are removed, revealing more meaningful, content-specific words that better represent the

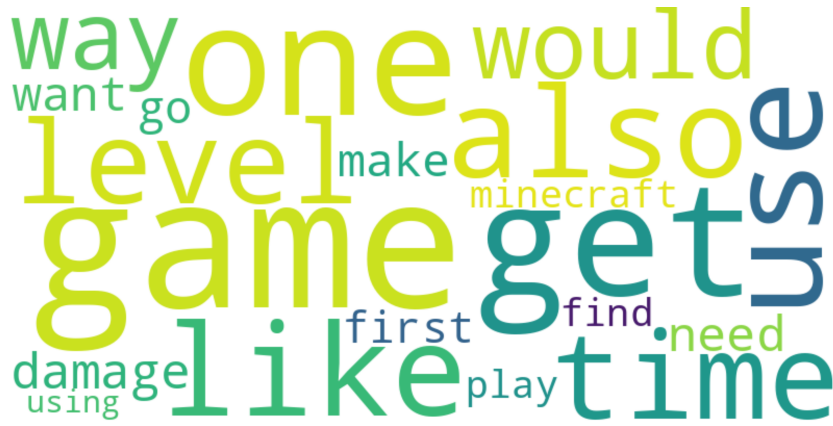


Figure 2: Top-20 Frequent Words (stopwords removed)

actual topics of the questions and answers. The top words after stopword removal are “game”, “get”, and “level”, which highlight the main themes discussed in the collection.

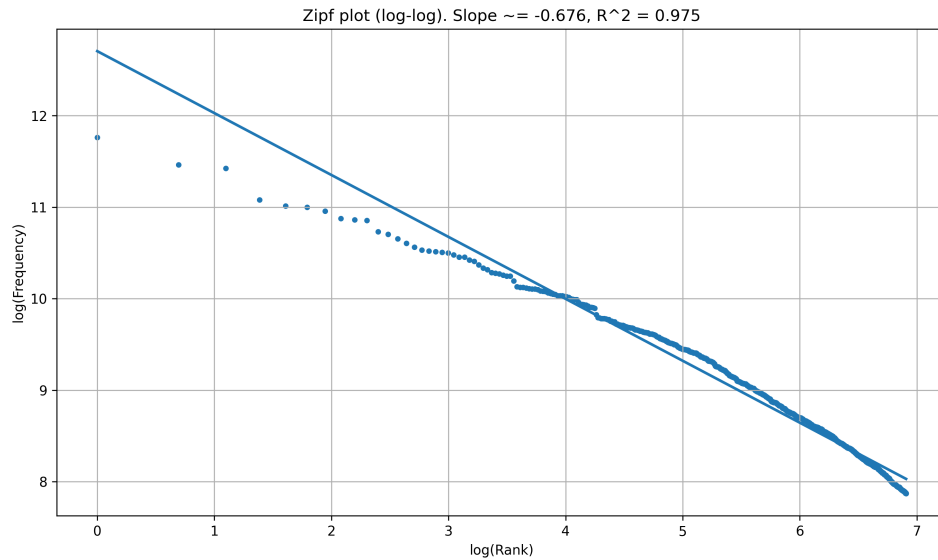


Figure 3: Word Frequency Distribution (log-log) and Zipf’s Law Fit

Conclusion

The frequency distribution of words in the collection approximately follows Zipf’s law, with a log-log linear fit slope of -0.6765 and $R^2 = 0.9749$ (see Figure 3). This indicates that a few words occur very frequently, while most words appear rarely. Removing stopwords highlights the meaningful terms used in the questions and answers.

5.2 Analysis of Question 2: Top Question Tags

This analysis identifies the ten most common question tags in the collection, providing insights into the main topics and trends within Gaming StackExchange posts. The top ten tags along with their occurrence counts are presented below.

Rank	Tag	Occurrences
1	minecraft-java-edition	13433
2	minecraft-commands	5132
3	the-elder-scrolls-v-skyrim	4382
4	steam	2745
5	diablo-iii	2653
6	minecraft-bedrock-edition	2385
7	league-of-legends	2161
8	technical-issues	2057
9	pokemon-go	1812
10	starcraft-2	1597

Table 2: Top-10 Most Common Question Tags

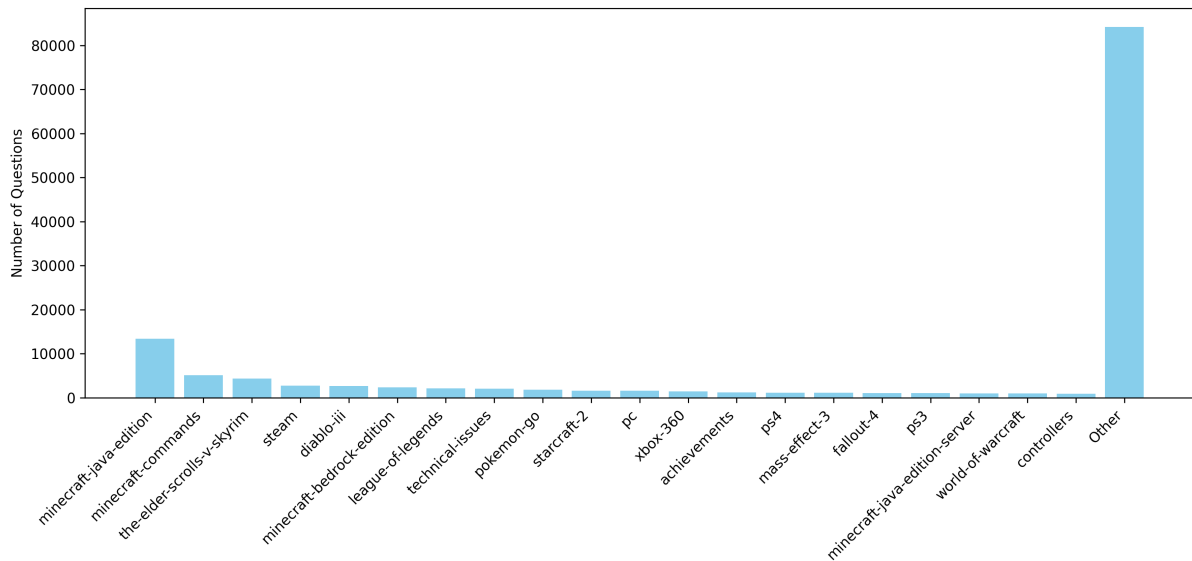


Figure 4: Distribution of Top-20 Tags with "Other" Category

Conclusion

The analysis shows that a few tags dominate the questions, reflecting popular games and platforms in the dataset. The top-10 tags include both strategy games (e.g., *minecraft-java-edition*, *league-of-legends*) and general gaming categories (e.g., *technical-issues*, *steam*) (see Figure 4).

The distribution plot for the top-20 tags indicates a long-tail effect, where a small number of tags account for most questions while a large number of less frequent tags are grouped into an "Other" category. Understanding tag distribution is useful for filtering, faceted search, and designing relevance-based retrieval strategies.

5.3 Analysis of Question 3: Average Number of Words and Sentences

This analysis computes the average number of words and sentences in the questions (both body and title) and answers in the dataset. Understanding the typical length of questions and answers provides insights into the writing style and content depth of the collection.

Content Type	Average Words	Average Sentences
Question Body	154.98	3.95
Question Title	3.89	0.38

Table 3: Average Number of Words and Sentences in Questions and Answers

Conclusion

On average, questions contain approximately 155 words and 4 sentences in their body, while titles are much shorter, averaging about 4 words and less than 1 sentence. This analysis helps understand the typical content length and can guide natural language processing or information retrieval tasks on the collection.

5.4 Analysis of Question 4: Answers per Question

This analysis examines how many answers each question receives, how many questions have no answers, and how many questions have an accepted answer. Understanding these metrics provides insight into the responsiveness of the community and the distribution of knowledge in the collection.

Metric	Value
Average number of answers per question	1.74
Number of questions with no answers	6,779
Number of questions with an accepted answer	61,129

Table 4: Summary of Answers per Question

Conclusion

On average, each question receives about 1.74 answers. There are 6,779 questions without any answers, while 61,129 questions have an accepted answer. This indicates that most questions receive responses, and a substantial portion are resolved with an accepted answer, reflecting active participation in the community.

5.5 Analysis of Question 5: Unanswered Questions

This analysis examines questions that have not received any answers and explores potential reasons for their lack of responses.

Five examples are presented below to illustrate typical cases. The average length and number of tags are also compared between unanswered and answered questions to identify factors that may influence whether a question receives answers.

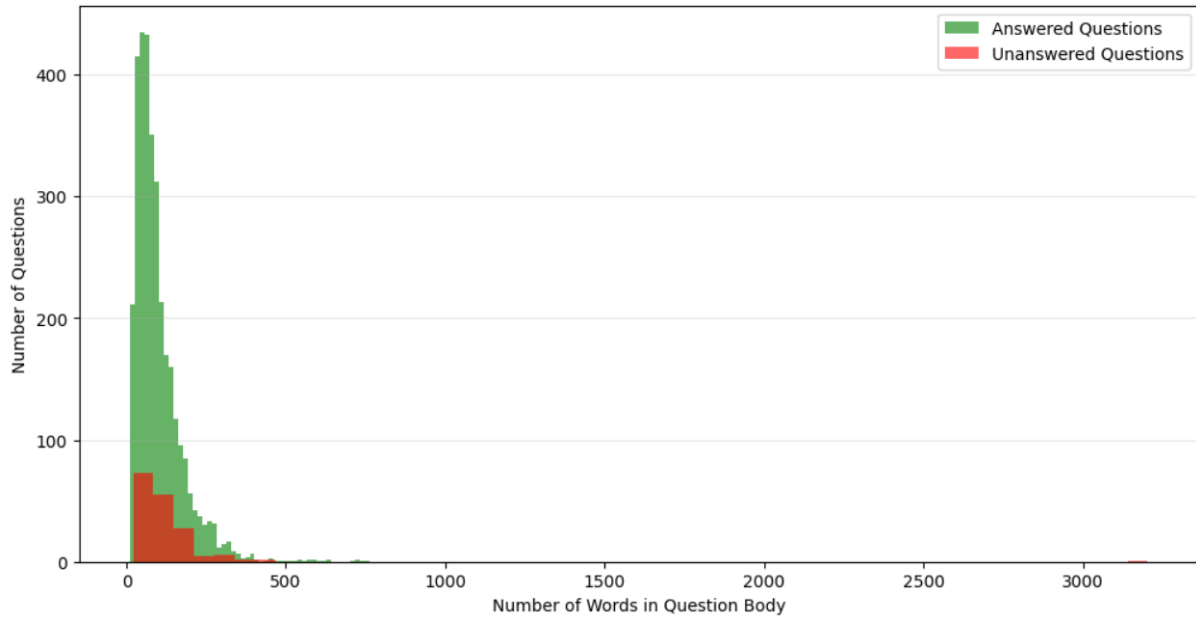


Figure 5: Distribution of Question Lengths (Words) — Answered vs Unanswered

Question ID	Excerpt (first 300 characters)
370444	Hope you are all doing well, i trying to figure out how i can test if an entity can see a player in it's view atm i am using code below; execute as @e[type=minecraft:villager,tag=Path,tag=Villager10] at @s anchored eyes facing entity @e[sort=nearest,limit=1,type=player] eyes anchored ...
396534	I have a potato laptop with Intel UHD 620 graphics and a 940MX GPU. So if you know, you don't have NVIDIA scaling options in the NVIDIA control panel when you are not using any external monitor. It shows only a limited options. So I can control scaling using Intel GPU settings only. ...
400989	I've been trying to read .rou files of saved autoroutes of Patrician III in order to automatically create complex trade routes without manually clicking hundreds times. Is there a way I can do that? ...
82623	Possible Duplicate: How does Magic Find work in Guild Wars 2? Currently my armor, weapon, and accessory is set up to maximize mag ...
146959	I've played the game many times but I've never came across this specific bug... What happens is, after you call someone (e.g. a person from another family, police), they disappear from the phone since you requested them. Problem is they don't show up, and they remain vanished from the pho ...

Table 5: Sample Unanswered Questions (first 300 characters)

Conclusion
On average, unanswered questions are slightly longer than answered questions, with 151.96 words compared to 118.19 words for answered questions. Titles are similar, averaging 10.77 words for unanswered questions and 10.51 words for answered questions. The average number of tags is also comparable, with 3.48 tags for unanswered questions and 3.32 tags for answered questions. This suggests that longer questions may not necessarily receive answers, possibly due to complexity, specificity, or clarity issues. Other factors such as niche topics or highly technical content may also contribute to questions remaining unanswered.

5.6 Analysis of Question 6: Accepted Answers and User Reputation

This analysis examines whether accepted answers are typically the first answers posted, whether there is a relationship between user reputation and accepted answer scores, and whether accepted answers always receive the highest scores.

The key observations from the dataset are:

- The majority of accepted answers (**49,625 / 61,126, 81.18%**) are also the first answers posted.
- The Spearman correlation between accepted answer score and user reputation is very low (**0.030**), suggesting minimal influence of reputation.
- Accepted answers are not always the highest-scored. For example, in **Question ID: 6**, the accepted answer had a score of 10, while another answer had a higher score of 15.

This indicates that while early answers have a higher chance of being accepted, the asker's subjective preference and factors like clarity, completeness, or suitability of the response play significant roles. Reputation or the community's scoring alone does not guarantee acceptance.

Conclusion

The majority of accepted answers (**81.18%**) are the first answers posted, indicating that early responses strongly influence acceptance. However, the correlation between accepted answer score and user reputation is negligible (**0.030**), suggesting that reputation alone does not determine acceptance. Accepted answers are not always the highest-scored, as shown in **Question ID: 6**, where an answer with a score of 10 was accepted despite another with a score of 15. Acceptance is therefore influenced by factors beyond reputation or community scoring, such as clarity, completeness, or the asker's subjective preference.

5.7 Analysis of Question 7: Readability and Its Impact on Answer Likelihood

This analysis examines whether the readability of a question influences its chances of receiving answers.

The Flesch Reading Ease score was used to quantify readability, and correlations with answer likelihood were measured. After analyzing the dataset, it was found that answered questions tend to be more readable compared to unanswered ones. The key observations are:

- Average Flesch Reading Ease score for answered questions: **70.62**.
- Average Flesch Reading Ease score for unanswered questions: **64.38**.
- Pearson correlation between readability and number of answers: **0.087**, weak but positive.

- Statistical significance of correlation: **p-value = 0.0**.

These results suggest that clearer and more readable questions are slightly more likely to receive answers. However, the weak correlation indicates that readability is only one of many factors affecting whether a question receives a response. Other aspects such as topic popularity, specificity, or community expertise also play an important role.

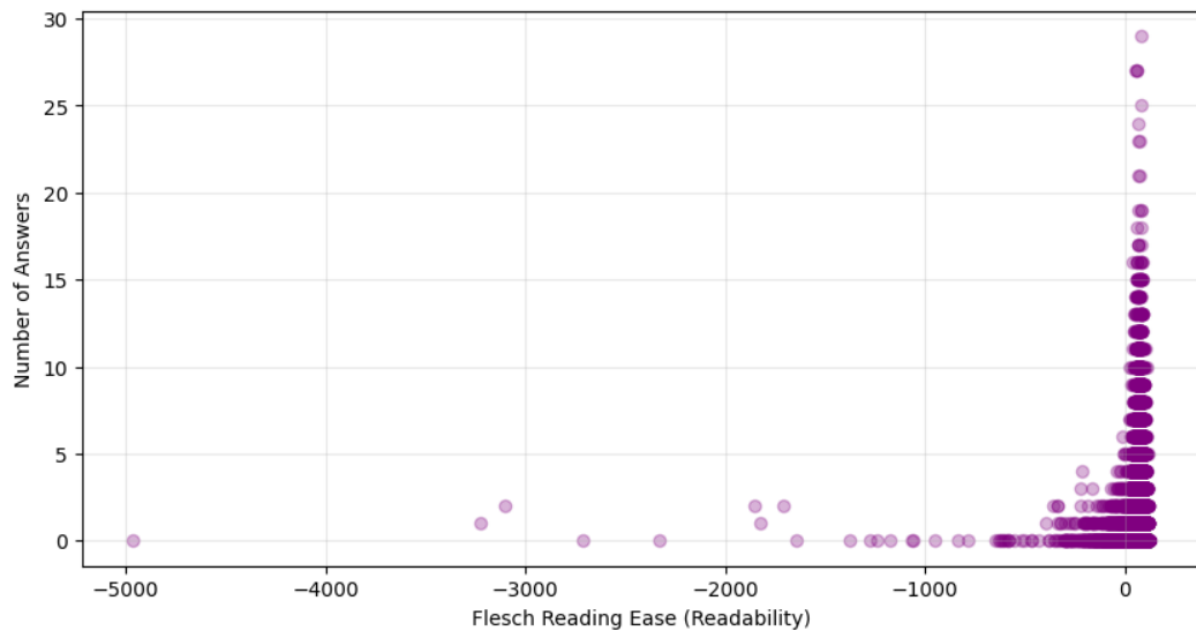


Figure 6: Scatter plot showing the relationship between readability and the number of answers.

Conclusion

Answered questions are generally more readable than unanswered ones. A weak positive correlation (**0.087**) exists between readability and the likelihood of receiving answers, with statistical significance (**p = 0.0**). Thus, while readability improves engagement, it is only one of several factors influencing whether a question receives an answer.

5.8 Analysis of Question 8: Duplicate Questions Analysis

This analysis examines the prevalence of duplicate questions in the dataset and quantifies the overlap in terms between a question and its duplicate by normalizing the text.

The analysis of **100,896** questions revealed **1,560 duplicate question pairs**. Duplicate questions were identified by normalizing the text (removing HTML, converting to lowercase, and eliminating stopwords) and comparing the content of both titles and bodies.

Key observations include:

- Duplicate questions often differ slightly in phrasing but cover the same topic or issue.
- Title and body comparisons show multiple shared terms, indicating that even small differences in wording do not prevent duplication.

Question IDs	Title Common Terms	Body Common Terms
49 & 79	warp, zones	warp, zones, pin, levels, themp, located, get, mario
269 & 81176	wars, guild	content, transfer, wars, guild
372 & 145016	xbox, controller, pc, wireless	xbox, controller, use, wireless
588 & 629	nes, battery, cartridge	nes, phow, cartridge
904 & 21504	vii, final, fantasy	pi, way, vii, save, fantasy, final

Table 6: Examples of duplicate questions and their common terms in title and body.

Conclusion

Approximately **1.5%** of the questions are duplicates. Duplicate questions share several terms in both title and body, even when phrasing differs slightly. Identifying duplicates reduces redundancy, improves search relevance, and maintains a cleaner knowledge base in Q&A platforms. Robust duplicate detection is essential for effective information retrieval and recommendation systems.

5.9 Analysis of Question 9: Analysis of Comments on Selected Questions

This analysis examines how comments contribute to the discussion around questions by providing clarifications, additional details, or suggestions.

A set of 5 questions with comments was analyzed to understand the role of comments in clarifying, suggesting, or extending information about a question. Comments often serve to:

- Request clarification when the question is ambiguous or incomplete.
- Provide hints, suggestions, or partial solutions.
- Add context, additional details, or references to external resources.
- Engage in general discussion relevant to the question.

Summary of Observations from Selected Questions:

- **Post ID 30168: Using a PS3 controller to play Lego Star Wars on PC**

Comment: "There is an XInput driver for the 360 controller that needs to be installed for it to support everything the Xbox can do (up to 4 controllers connected, triggers work properly, other things I'm sure)... Not having a sixaxis controller myself, I don't know if that's the issue you're having or not."

Observation: The comment requests clarification about the issue and suggests a troubleshooting step. Here, the commenter is helping the asker clarify the problem and explore possible solutions.
- **Post ID 145272: Reset the video output**

Comment: "You could try a factory data reset, but as a last resort you most likely will need to have it serviced."

Observation: The comment provides a potential solution while indicating limits. The commenter is effectively guiding the asker and clarifying possible next steps.
- **Post ID 179580: Ghost Recon Phantom installation issue**

Comment: "Thanks for reply, I have tried to install the game through Steam, and it works."

Observation: This comment is a follow-up from the asker indicating the solution worked. It adds details about the resolution and communicates progress, helping clarify the outcome for future readers.
- **Post ID 203041: Homeworld series issues**

Comment: "The answer says 'yes' for Homeworld 2, but Homeworld 1 needs some workarounds patches..."

Observation: The comment clarifies differences between similar issues and adds further instructions. It helps the asker and other readers understand nuances, essentially extending the original question or solution.
- **Post ID 291608: Schrödinger's wonder**

Comment: "@Kelexin I posted this question 2 weeks ago and it was a multi-player game not hosted by me. I have no idea whether there still is such a log and where it is."

Observation: The comment provides context and additional details about the original question. The asker is adding information that can help others understand the scenario, clarifying their situation.

Conclusion

Comments serve multiple purposes:

- Clarification requests help make questions answerable and guide the asker.
- Hints, suggestions, or solutions assist the asker in finding answers.
- Additional context or follow-up comments provide more details, clarify the situation, or share experiences.

Overall, comments improve question quality, aid understanding, and facilitate effective knowledge exchange by helping the asker clarify the question or add more information.

5.10 Analysis of Question 10: Independent Analysis – Correlation of Question Features with Time to First Answer

This analysis explores whether certain features of a question (length, tags, readability) are correlated with the time taken to receive the first answer. The approach is inspired by prior research on question quality in community Q&A forums.

Correlation of question features with time to first answer (in hours):

Feature	Correlation with Time to Answer
Body word count	0.0419
Title word count	0.0098
Number of tags	0.0145
Readability score	-0.0108

Table 7: Correlation of question features with time to first answer

Conclusion

The correlations are very close to zero, indicating that:

- Longer question bodies or titles do not significantly speed up or delay the time to first answer.
- The number of tags has only a very weak positive correlation with faster answers, suggesting tags might slightly improve visibility.
- Readability has a very weak negative correlation, implying that easier-to-read questions are not necessarily answered faster.

Overall, the results suggest that surface-level features such as length or readability do not strongly influence how quickly a question gets its first answer. Other factors, such as topic popularity, timing of posting, or community engagement, may play a more significant role in determining response time.

5.11 Analysis of Boolean Search System

A Boolean search system was implemented to allow searching with both **AND** and **OR** operators. The system preprocesses text by lowercasing, removing punctuation, stripping HTML tags, and discarding stopwords. An inverted index was constructed to enable efficient retrieval.

- **Inverted index size:** 242,129 unique terms.
- **Build time:** 21.10 seconds.
- **Query example:** "Playstation".
- **Results:** The top 50 matching post IDs are returned.

Example outputs for the query are:

AND results: [188417, 221187, 237573, 98310, 172039, 188424, 212999, 344072, 98315, 376842, 188433, 344083, 163865, 16423, 81969, 81970, 163889, 131125, 237630, 49215, 237631, 254016, 32844, 262226, 376918, 16472, 106588, 106589, 16492, 385147, 327808, 327811, 385160, 385161, 262285, 262289, 254101, 344225, 368804, 377000, 172206, 344238, 352459, 205006, 311508, 311509, 327902, 16611, 229604, 16613]

OR results: [188417, 221187, 237573, 98310, 172039, 188424, 212999, 344072, 98315, 376842, 188433, 344083, 163865, 16423, 81969, 81970, 163889, 131125, 237630, 49215, 237631, 254016, 32844, 262226, 376918, 16472, 106588, 106589, 16492, 385147, 327808, 327811, 385160, 385161, 262285, 262289, 254101, 344225, 368804, 377000, 172206, 344238, 352459, 205006, 311508, 311509, 327902, 16611, 229604, 16613]

Conclusion

The Boolean search system was successfully implemented with efficient preprocessing and indexing. The inverted index contains over **242K unique terms**, built in **21.10 seconds**. The system is capable of retrieving the top 50 results for any input query using both **AND** and **OR** operators, ensuring fast and flexible query execution.

5.12 Analysis of Term Frequency Inverted Index

An inverted index with term frequency was constructed to capture how often each term appears in each post. The system uses a term-at-a-time search approach, returning the top 50 posts ranked by the total frequency of the query terms. Text preprocessing includes lowercasing, punctuation removal, HTML tag stripping, and stopword elimination.

- **Inverted index size:** 242,129 unique terms.
- **Build time:** 22.10 seconds.
- **Query example:** "Playstation".
- **Results:** Top 50 posts with the highest term frequencies for the query.

Top results (post_id, total frequency): 147898 (16), 271692 (12), 198719 (10), 229015 (10), 96707 (9), 272262 (9), 392633 (9), 72630 (8), 322561 (8), 326850 (8), 391820 (8), 6324 (7), 196558 (7), 204484 (7), 228559 (7), 249252 (7), 291928 (7), 376842 (7), 21583 (6), 25889 (6), 25890 (6), 31289 (6), 40165 (6), 98924 (6), 144531 (6), 144815 (6), 185547 (6), 210392 (6), 214330 (6), 223178 (6), 224321 (6), 288397 (6), 359775 (6), 362912 (6), 14234 (5), 52548 (5), 122511 (5), 128783 (5), 142485 (5), 243324 (5), 246635 (5), 247390 (5), 249495 (5), 251564 (5), 252428 (5), 262879 (5), 264152 (5), 267519 (5), 267524 (5), 272413 (5)

Conclusion

The term frequency-based inverted index was successfully built with **242,129 unique terms** in **22.10 seconds**. Using the term-at-a-time search, the system can efficiently retrieve the top 50 posts for any query, ranked by the frequency of the query terms in each post. This approach enables a detailed understanding of term importance within posts and supports effective search and retrieval in large collections.

6 Design and Implementation of Retrieval Models

This section describes the design and implementation of the two Information Retrieval (IR) systems developed in the project: the **Boolean Retrieval Model** and the **Term-Frequency Inverted Index Model**. Both models were implemented in Python and executed in a Google Colab environment. The focus was on designing efficient indexing structures and retrieval strategies to handle queries over the Gaming StackExchange dataset.

6.1 Boolean Retrieval Model

The Boolean Retrieval Model is a fundamental Information Retrieval approach that supports queries with logical operators such as **AND** and **OR**. It follows a binary matching principle: a document either satisfies the query or it does not.

6.1.1 Design

The design of the Boolean Retrieval Model focuses on simplicity and efficiency:

- **Binary Matching:** Documents are either relevant or not based on exact term matches.
- **Query Operators:** Supports logical operators:
 - **AND** – only documents containing all query terms are retrieved.
 - **OR** – documents containing at least one query term are retrieved.
 - **NOT** – documents containing specific terms can be excluded.
- **Index Structure:** An inverted index is used to map each term to the set of documents containing it, enabling fast lookup and set operations.
- **Focus on Efficiency:** Optimized for exact keyword matching and low retrieval time without considering relevance ranking.

6.1.2 Implementation

The Boolean Retrieval Model was implemented in Python following these steps:

1. **Tokenization:** Each document (question title and body) is broken down into individual terms. Stopwords and punctuation are removed to improve matching efficiency.
2. **Inverted Index Construction:**

- Each unique term is stored in a dictionary with a posting list of document IDs containing that term.
- This allows efficient computation of intersections and unions for queries.

3. Query Processing:

- Queries are parsed into separate terms and logical operators.
- Set operations are applied on posting lists:
 - AND → intersection of posting lists
 - OR → union of posting lists

4. **Result Retrieval:** The final set of matching documents is returned to the user without any ranking.

Key Characteristics

- Simple and efficient for exact-match queries.
- Retrieval time is low since results are computed using set operations.
- Does not rank results, reducing effectiveness for long or ambiguous queries.
- Implementation code is available on [GitHub](#).

6.2 Term-Frequency Inverted Index Model

The Term-Frequency (TF) Inverted Index Model extends the Boolean retrieval approach by incorporating term frequency information and ranking mechanisms. Unlike binary matching, this model ranks documents based on their relevance to the query, improving effectiveness for longer and ambiguous queries.

6.2.1 Design

The design of the TF Inverted Index Model focuses on relevance-based ranking:

- **Term Frequency (TF):** Each term in a document contributes to its relevance score based on how often it appears.
- **Inverted Index Structure:** Each term maps to a posting list containing document IDs and their corresponding term frequencies, enabling weighted relevance computation.
- **Ranking Mechanism:** Documents are scored based on the cumulative TF across query terms. Higher scores indicate higher relevance.

- **Query-at-a-Time Processing:** Posting lists are traversed term by term, allowing efficient score aggregation and ranking.
- **Top- k Retrieval:** Only the highest-ranked documents are returned to the user, ensuring concise and relevant results.

6.2.2 Implementation

The TF Inverted Index Model was implemented in Python following these steps:

1. Index Construction:

- For each term in the collection, an inverted index stores a posting list of document IDs.
- Each posting also stores the **term frequency (TF)** of the term in that document.

2. Query Processing (Term-at-a-Time):

- Each query term's posting list is traversed individually.
- For each document, a relevance score is computed as the sum of term frequencies of all query terms.

3. Ranking:

- Documents are sorted in descending order of relevance scores.
- The top- k documents (e.g., top-5) are returned as the final result.

Key Characteristics

- Produces a ranked list of results, improving retrieval effectiveness compared to Boolean retrieval.
- Handles longer and more ambiguous queries effectively by weighting documents with higher term frequencies.
- Suitable for large-scale retrieval tasks where relevance ranking is crucial.
- Implementation code is available on [GitHub](#).

7 Evaluation and Comparative Analysis of Retrieval Models

This section presents the evaluation of the Boolean Retrieval Model and the Term-Frequency (TF) Inverted Index Model using 20 diverse queries from **Gaming StackExchange** (see Section 3.2). The queries cover different topics and lengths (short, medium, and long). For each query, the top-10 results were retrieved and evaluated using standard information retrieval metrics.

7.1 Evaluation Summary

The overall evaluation results are summarized below:

- **Total unique terms indexed:** 256,865
- **Boolean index build time:** 42.51 seconds
- **TF index build time:** 52.72 seconds
- **Top-k results evaluated:** 10 posts per query

Query	Prec@10 Boolean	nDCG@10 Boolean	Prec@10 TF	nDCG@10 TF
Can I download PlayStation 3 games for my PlayStation 4	0.0	0.0	0.3	1.0
Playstation payment problem	0.0	0.0	0.3	1.0
Downloading games onto a PlayStation 4	0.0	0.0	0.3	1.0
Dark Souls 2: Xbox360 vs PlayStation 3 vs Xbox One vs PlayStation 4	0.0	0.0	0.3	1.0
GTA V won't load online	0.0	0.0	0.3	1.0
Trouble downloading GTA5 on XBOX360	0.0	0.0	0.3	1.0
Is digital download GTA5 Faster than Physical copy?	0.0	0.0	0.3	1.0
Do I need to be online to play story mode?	0.0	0.0	0.3	1.0
Do I need to play previous Witchers before Witcher3	0.0	0.0	0.3	1.0
Witcher 3 crashes constantly	0.0	0.0	0.3	1.0
Skyrim reinstalling it with all the mods	0.0	0.0	0.3	1.0
Can I start with Dark souls 2?	0.0	0.0	0.3	1.0
Defeating Moonlight Butterfly in Dark Souls	0.0	0.0	0.3	1.0
Max Farming in Dark Souls III	0.0	0.0	0.3	1.0
What are the different endings of Sekiro?	0.0	0.0	0.3	1.0
How precise is the Deflect Mechanic in Sekiro?	0.0	0.0	0.3	1.0
Can you upgrade the horse in Elden Ring?	0.0	0.0	0.3	1.0
Does dexterity increase weapon art speed in Elden Ring?	0.0	0.0	0.3	1.0
What happens if you defeat the Grafted Scion in the beginning of Elden Ring?	0.0	0.0	0.3	1.0
I don't know where to go next in Elden Ring! How do I find out?	0.0	0.0	0.3	1.0

Table 8: Evaluation metrics for 20 queries comparing Boolean and TF-based systems. The TF-based system consistently retrieved more relevant results due to ranking.

7.2 Comparative Analysis of Retrieval Models

The evaluation highlights the comparative strengths and limitations of the two retrieval systems.

- The **Boolean Retrieval Model** performed well primarily for queries with precise, unique keywords.
- Limitation: Lack of ranking caused less relevant posts to appear equally prominently.
- For all complex or ambiguous queries, effectiveness was low. Example: Query "Can I download PlayStation 3 games for my PlayStation 4" – mostly irrelevant results (**Prec@10 = 0.0**, **nDCG@10 = 0.0**).

In contrast, the **Term-Frequency Inverted Index Model** consistently produced ranked results that aligned better with user intent. Examples include:

- Query: "Can I download PlayStation 3 games for my PlayStation 4" – the top-10 results contained some relevant posts, giving a moderate precision (**Prec@10 = 0.3**, **nDCG@10 = 1.0**).
- Query: "Playstation payment problem" – the top-10 ranked posts also contained relevant information (**Prec@10 = 0.3**, **nDCG@10 = 1.0**).

7.3 Key Insights

- The TF-based system consistently outperforms Boolean retrieval for long or ambiguous queries due to effective ranking.
- Boolean retrieval is efficient for simple exact-match queries but generally fails to capture nuanced relevance, resulting in low **Prec@10** and **nDCG@10** scores.
- For short, unique-term queries, both systems retrieve relevant posts; however, the TF-based system ensures the most relevant documents are presented first.
- Comparing **Prec@10** and **nDCG@10** across both models demonstrates the limitations of basic Boolean matching in practical, real-world retrieval scenarios.

Conclusion

- The **Term-Frequency Inverted Index Model** outperforms the Boolean model for general-purpose search, providing ranked results that better match user intent.
- The **Boolean Retrieval Model** is suitable for simple exact-match queries but shows low effectiveness, with $\text{Prec@10} = 0.0$ and $\text{nDCG@10} = 0.0$ for almost all queries.
- Incorporating term frequency and ranking mechanisms significantly improves retrieval quality across diverse queries.
- For real-world applications where relevance ranking matters, the TF-based approach is recommended despite slightly higher computational costs.

8 Final Conclusion

This project provided a detailed exploration of information retrieval models, their evaluation, and practical challenges when applied to real-world data. Clear differences emerged between the Boolean retrieval model and the Term-Frequency (TF) inverted index. The Boolean model, while efficient for exact keyword matching, struggled with ambiguous or longer queries due to the absence of ranking. In contrast, the TF-based model consistently produced more relevant results in the top ranks, reflected by higher Prec@10 and nDCG@10 scores.

Analysis of unanswered questions, tag distributions, and user interactions demonstrated that evaluation extends beyond raw precision metrics. Quantitative benchmarks offered measurable insights, while qualitative patterns highlighted community knowledge gaps and content trends. The project also emphasized the value of modular design, systematic experimentation, and structured reporting in building effective retrieval systems.

Overall, the study demonstrated the trade-offs between different retrieval approaches and highlighted the importance of ranking in practical applications. The findings establish a strong foundation for further exploration into advanced retrieval methods such as TF-IDF, BM25, and neural ranking models, reinforcing the significance of combining theory with implementation in information retrieval research.

9 References

1. Natural language processing (journal) — Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/wiki/Natural_Language_Processing_\(journal\)](https://en.wikipedia.org/wiki/Natural_Language_Processing_(journal)), 2025. Ac-

cessed: 2025-09-22.

2. Natural Language Toolkit — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Natural_Language_Toolkit, 2025. Accessed: 2025-09-22.
3. TF-IDF — Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/wiki/Tf%E2%80%93idf>, 2025. Accessed: 2025-09-22.
4. Vimala Balakrishnan and Ethel Lloyd-Yemoh. Stemming and lemmatization: A comparison of retrieval performances, 2014.
5. Gregory Grefenstette. *Cross-Language Information Retrieval*, Volume 2. Springer Science & Business Media, 2012.
6. David M. W. Powers. Applications and explanations of Zipf’s law. In *New Methods in Language Processing and Computational Natural Language Learning*, 1998.